

ИССЛЕДОВАНИЕ

11 НОЯБРЯ 2025 ГОДА

BYO SWE-grep: автоматическое обучение сверхбыстрых поисковых субагентов на основе вашей базы знаний (часть 1)

Поисковые субагенты, обученные с помощью RL, изучают структуру вашей базы знаний для быстрого и надежного поиска



Чарльз О'Нил (Базеттен)



Джонатон Лю (Бастетен)

Недавно компания Windsurf [объявила](#), что обучила квазифундаментальные модели, которые выступают в роли субагентов, способных очень быстро находить нужные фрагменты кода (общая цель состояла в том, чтобы помочь большой модели для написания кода, которая вызывала этих субагентов для поиска). Ранее мы обучали множество моделей для поиска в контекстах, не связанных с программированием, например, когда клиентам нужна модель, способная искать в базе знаний гораздо более гибким способом, чем статическая RAG, с более широким охватом, модель, которая действительно хорошо понимает их базу знаний. Представляем наш собственный обучающий продукт *Search Subagents*, доступный для текущих клиентов Baseten и вскоре появится на нашей платформе.

Несмотря на то, что наш инструмент *Search Subagents* также позволяет напрямую обучать модель с помощью RL на наборе данных для оценки (совокупность подсказок для LLM в качестве судьи, часто оптимизируемых с помощью [Lumina](#)), в этой статье мы сосредоточимся на возможностях поиска. Мы часто рекомендуем использовать методы, не связанные с RL, такие как [iSFT](#) и [RGT](#), для обучения модели составлению окончательного ответа после поиска, а не с помощью RL, так как это требует слишком больших вычислительных ресурсов.

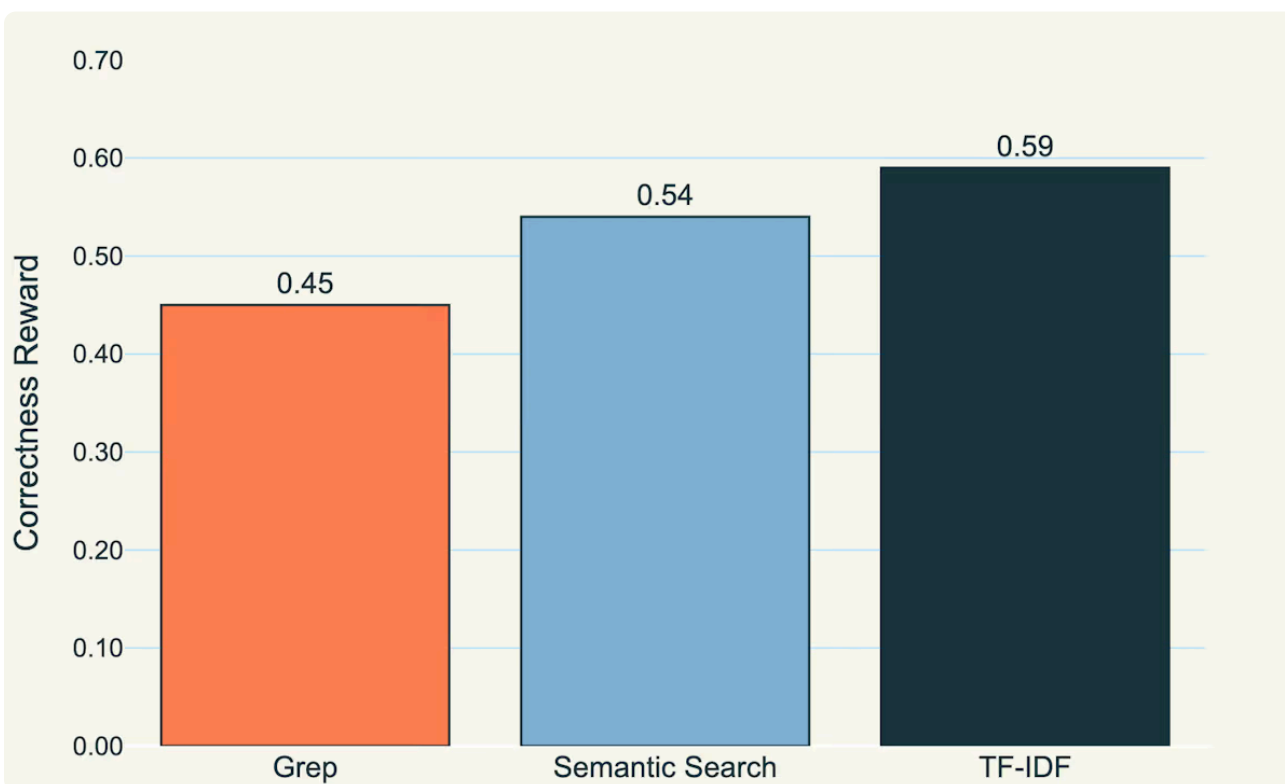
Чем отличается поиск в базе знаний

Базы знаний в регулируемых отраслях — сущий кошмар для традиционного поиска. У вас есть тысячи страховых полисов, клинических рекомендаций и нормативных документов, которые связаны между собой так, что понять эту взаимосвязь могут только эксперты в данной области. RAG здесь не справляется, потому что поиск осуществляется на основе поверхностного сходства, а не реального понимания. Он находит совпадения по ключевым словам, но пропускает важные перекрестные ссылки на три других документа, которые на самом деле отвечают на вопрос. А в сфере здравоохранения и страхования ошибка может привести не только к бесполезным результатам, но и к катастрофическим последствиям с точки зрения ответственности.

Если для кода Windsurf решил эту проблему с помощью SWE-grep, то корпоративные базы знаний — это совсем другая история. База знаний каждого клиента имеет свою уникальную структуру, терминологию и сеть взаимосвязей, которые формировались годами. Мы пришли к выводу, что нужна модель, которая действительно понимает эту конкретную базу знаний, а не просто универсальный поисковый инструмент. И что немаловажно, для этого не нужна огромная модель, поэтому мы можем сделать ее быстрой.

TF-IDF: недооцененный средний путь

Мы несколько месяцев экспериментировали с различными поисковыми алгоритмами и пришли к неожиданному выводу: для этой задачи оптимален алгоритм TF-IDF (Term Frequency — Inverse Document Frequency, частота термина — обратная частота документа). С одной стороны, есть инструменты для точного соответствия, такие как grep, — они быстрые, но негибкие. С другой стороны, есть семантические векторные представления, которые гибкие, но непрозрачные и часто дают неоднозначные результаты. TF-IDF находится где-то посередине: он обрабатывает синонимы и частичные совпадения, но при этом достаточно интерпретируем, чтобы пользователи могли понять, почему были найдены те или иные документы.



Сравнение различных базовых поисковых инфраструктур для поискового инструмента, который мы разработали для Claude Sonnet 4.5. TF-IDF лучше всего работает «из коробки», а также является самым быстрым с точки зрения совокупной производительности и общего времени выполнения.

Настоящее преимущество становится очевидным во время обучения с подкреплением. TF-IDF обеспечивает детерминированную оценку, а значит, стабильные сигналы вознаграждения. Эмбединги слишком нестабильны: небольшие изменения в запросе могут привести к резким скачкам в количестве найденных документов, из-за чего модель практически не может научиться использовать последовательные стратегии поиска. Благодаря предварительно созданным и постоянно обновляемым индексам TF-IDF поиск выполняется менее чем за 100 мс даже в огромных базах знаний, при этом сохраняется гибкость, необходимая для обработки различных вариантов запросов.

Конечно, инструменты поиска на основе грег зачастую гораздо лучше подходят для написания кода. К счастью, сам механизм поиска практически не зависит от остальной части конвейера: что бы вы ни дали модели, она научится оптимизировать процесс. Со временем мы позволим пользователям выбирать метод поиска, который они хотят использовать, а также будем давать рекомендации на основе характеристик их базы знаний.

Synthetic Data: The Zero-Shot Training Pipeline

The really cool part (at least to us) is that we generate all the training data synthetically. A customer drops in their knowledge base, and we automatically create a complete training dataset without needing any real user queries. This works because retrieval is fundamentally a verifiable task; there's deterministic ground truth for what should be retrieved for any given query.

The pipeline starts with raw markdown documents that get chunked into sections with deterministic chunk IDs. We filter these for quality ie checking word count, text quality scores, and detecting chunks that are mostly tables. For each high-quality chunk, we use frontier models to generate realistic customer questions that would be answered by that chunk. The key is using domain-specific example questions to guide the conversational style. Insurance questions sound different from healthcare questions, and the synthetic data needs to reflect that.

We then verify each synthetic question-answer pairs by actually running it through our search tool. If the target chunk ID appears in the top search results, we keep it. This verification loop ensures we're only training on examples where the search tool could plausibly find the right answer. The final dataset contains the synthetic question, the chunk ID as the answer, and the chunk content as reference text. This is, we emphasize, completely automated, with no human annotation required.

Training with AIPO and Partial Credit Rewards

We train these customer-specific models with rollouts using a token-level Advantage-Indexed Policy Optimization (AIPO) objective, optimizing for both retrieval accuracy and efficiency. The reward is composite: 90% weight on answer correctness and 10% on *retrieval partial credit*.

The retrieval partial credit reward scans all messages in the completion and parses any tool calls the model makes. If the target chunk ID appears anywhere in those search results (at any turn, even if the final answer is wrong) the rollout earns partial credit. This is especially helpful early in training because it provides learning signal before the model has reliably learned to compose answers from retrieved text. The model may issue multiple searches across 2–3 turns, and we consider results from all of them.

For the update, we use an off-policy REINFORCE estimator with clipped importance weights and leave-one-out advantages computed upstream. We typically sample $K = 8$ rollouts per prompt; for rollout (i), the advantage is the rollout's reward minus the mean reward of the other (K-1) rollouts. To stabilize training, we (i) apply token-level AIPO clipping of the importance ratio, (ii) mask malformed tool-call spans via the loss mask, and (iii) optionally apply additional ratio safety (log-ratio clamp and masking of tokens/sequences outside configured bounds) and length normalization.

Formally, letting μ be the behavior policy that generated the rollout and π the current policy, the (maximization) objective is

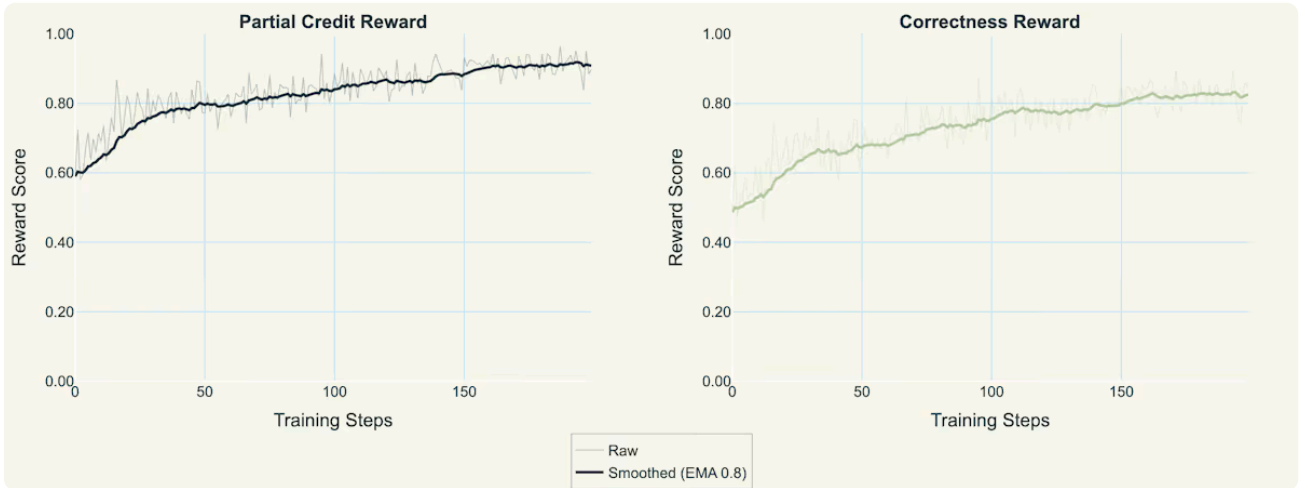
$$J_{\text{AIPO}}(\theta) = \frac{1}{\sum_{j=1}^N \sum_{i=1}^G |y_i^{(j)}|} \sum_{j=1}^N \sum_{i=1}^G \sum_{t=1}^{|y_i^{(j)}|} \min \left(\underbrace{\frac{\pi(y_{i,t}^{(j)} | x_j, y_{i,<t}^{(j)})}{\mu(y_{i,t}^{(j)} | x_j, y_{i,<t}^{(j)})}}_{\text{importance ratio } r_{i,t}^{(j)}}, \delta \right) \hat{A}_{i,t}^{(j)}$$

where $\hat{A}_{i,t}^{(j)}$ is the token-level advantage and δ is the clipping threshold. With leave-one-out baselines,

$$A_i = R(\tau_i) - \frac{1}{K-1} \sum_{k \neq i} R(\tau_k)$$

and in practice $\hat{A}_{i,t}^{(j)}$ denotes the per-token credit assignment for rollout (i) (optionally broadcast per sequence when using sequence-ratio variance reduction). We omit entropy and reference-KL terms from the objective; they are monitored as diagnostics only.

In practice, we typically use Qwen models up to 4B parameters for the search subagent, determined by the size and complexity of the knowledge base. Here, we show the reward curves for **qwen3-4b** over 200 steps, using a LoRA with rank 32.

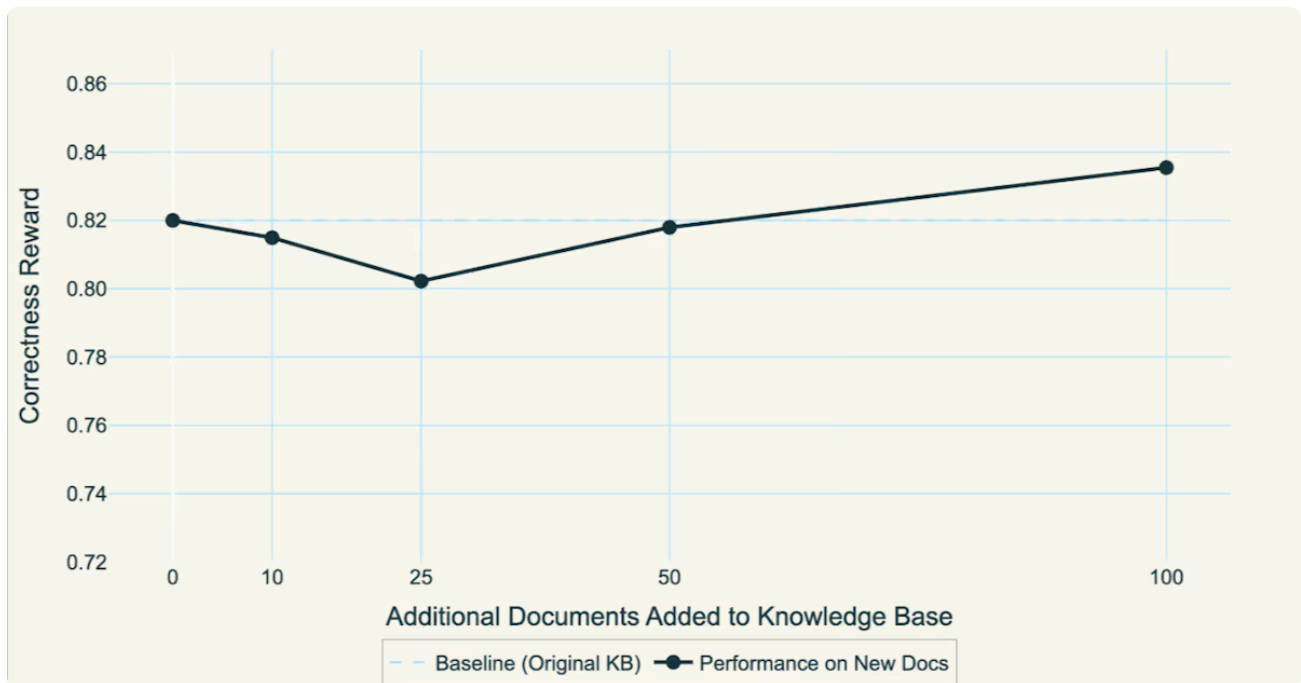


Reward curves for qwen3-4b on the insurance knowledge base.

Generalization: Models That Actually Learn the Domain

The most exciting result is that these models genuinely learn the structure and patterns of the knowledge base. When customers add new documents after training, the model maintains its performance without retraining. It has learned how insurance policies are

structured, how clinical guidelines reference each other, how regulatory documents are organized, rather than memorized specific document-query pairs.

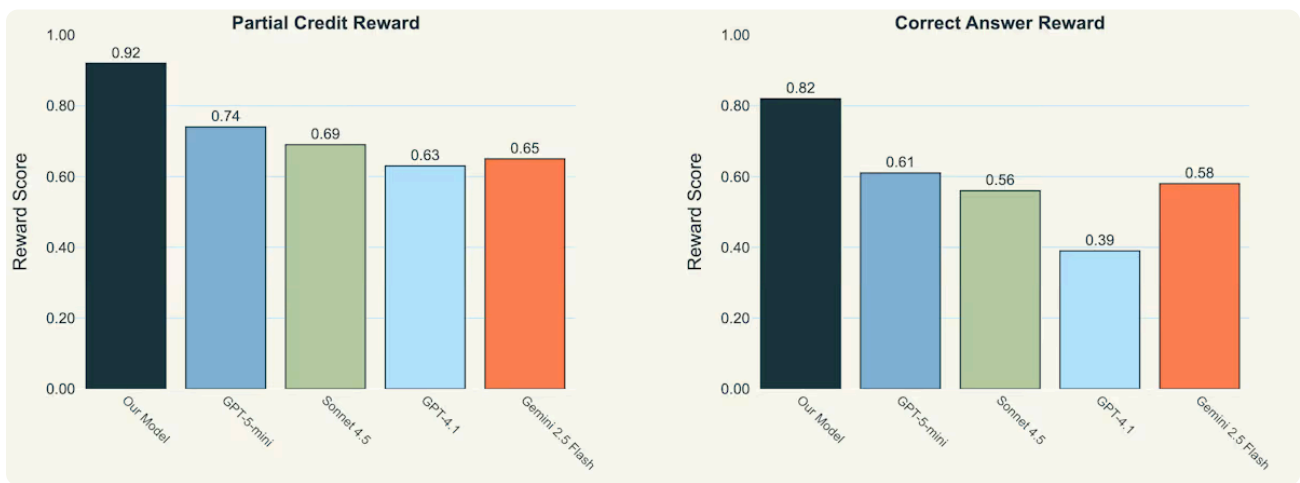


Trained model F1 score when we freeze the weights and simply test the F1 score with queries on documents not seen during training.

This generalization is what separates our approach from traditional fine-tuning. We're not teaching the model a mapping from questions to documents. We're teaching it to understand the domain itself, that is, the terminology, the document structures, the types of questions users ask, and how information is organized. Once it learns these patterns, it can apply them to new documents seamlessly.

Real-World Performance

On our internal insurance and healthcare evaluation sets, domain-specific *Search Subagents* achieve significantly higher partial and complete correctness scores on retrieval tasks, all while being significantly faster. Compare this to standard RAG at 0.42 F1, or GPT-4 with RAG at 0.61 F1 but taking 2.8 seconds. The speed matters as much as the accuracy e.g. when an insurance adjuster is on the phone with a claimant, they need answers in milliseconds, not seconds. We are currently optimizing our *Search Subagents* with Baseten's inference stack to run as quickly as possible, and will do full latency benchmarking in the future.



Performance of our final trained model on the insurance knowledge base compared with closed-source models given access to the same tool and instructions. Interestingly, our base model actually starts relatively on-par with some of the other models.

A major insurance provider deployed this across their page policy database. Previously, their support agents relied on fragile, static RAG systems searching through documents for specific exclusions or coverage details. Now they get precise answers with cross-references in under half a second. The model understands that when someone asks about “water damage in coastal properties with previous flood history,” they need the specific exclusion clause, the underwriting guidelines, and the claims history impact tables, which can all be retrieved in parallel in a single search.

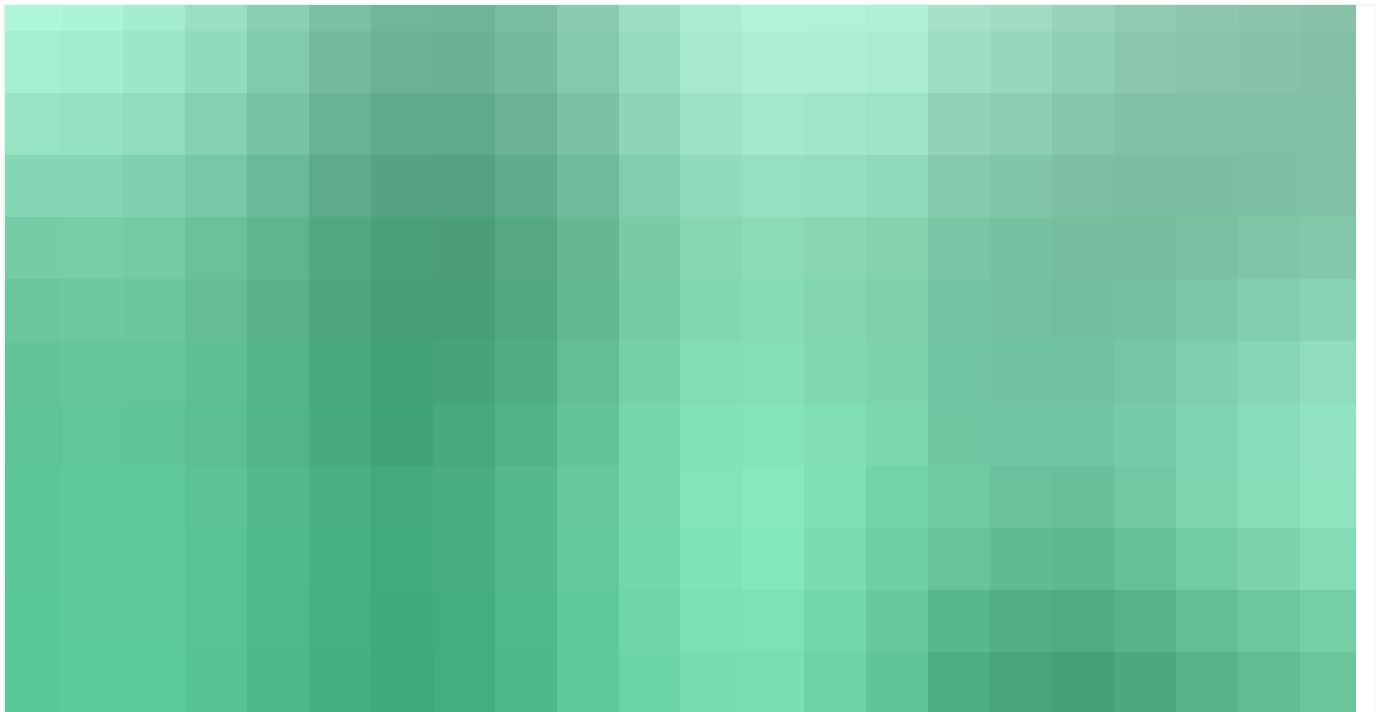
Future Directions and Availability

We’re exploring several extensions to make these models even more powerful. Adaptive parallelization would let models dynamically adjust their parallel search budget based on query complexity, where simple questions get one search, complex multi-part questions get many. This is already naively built into our pipeline but we want to make it more explicit in the reward functions. We also want to allow people to choose the backend of the search tool, as different search methodologies are sometimes more useful (ie grep for coding knowledge bases), and possibly even training the model to use different search methods simultaneously.

Search Subagents training is available now to existing Baseten customers and will be generally available in our platform soon. The entire pipeline from knowledge base ingestion to trained model deployment is fully automated. What makes this particularly powerful is the true zero-shot nature; you literally just drop in your knowledge base, and we generate the synthetic training data, train the model with RL, and deploy a domain-specific search agent that intimately understands your documents. No real user queries, no manual annotation, just pure automated intelligence generation from your existing knowledge.

The combination of synthetic data generation, TF-IDF indexing, and GRPO training creates models that are fast, accurate, and most importantly, actually understand the domains they're searching. They're a fundamentally different approach from RAG to knowledge base search that learns and adapts to each customer's unique information architecture.

Other research



RESEARCH

Post-Training Science for Supervised Fine-Tuning

How the optimal learning rate and batch size move with model scale, family, and data, and whether one selection rule transfers across them.



CHARLES O'NEILL • 2 others



RESEARCH

Still: Amortized KV Cache Compaction in a Single Forward Pass

You can shrink a language model's KV cache by 200×, in a single forward pass, and it still answers correctly.



RESEARCH

Post-training frontier legal agents with Baseten Research

Post-training a 27B open-weight model to the frontier on Harvey LAB with Baseten Research



CHARLES O'NEILL • 5 others

Explore Baseten today

START DEPLOYING

TALK TO AN ENGINEER >

Product

Dedicated Inference

Model APIs

Training

Frontier Gateway

Baseten Inference Platform

Model Runtimes

Infrastructure

Multi-cloud

Deployment options

Cloud

Self-hosted

Hybrid

Embedded engineering

Forward deployed engineers

Modalities

- Transcription
- Image Generation
- Text-to-speech
- Large language models
- Compound AI
- Embeddings

Industries

- Enterprise
- Healthcare

Developer

- Model library
- Documentation
- Changelog

Resources

- Research
- Customers
- Blog
- Guides
- Events
- Partners
- Savings calculator
- Trust Center
- About us
- Startups
- Careers
- Contact us

Popular models

- GLM 5.2
- Kimi K2.7 Code
- DeepSeek V4
- GPT OSS 120B
- Whisper Large V3
- NVIDIA Nemotron 3 Ultra
- Explore all

Legal

- Terms and Conditions
- Privacy Policy
- Service Level Agreement

● ALL SYSTEMS NORMAL



